Tugas Besar

IF4031

Arsitektur Aplikasi Terdistribusi

Milestone 1

Waktu Mulai

Rabu, 16 September 2026, 17:00 WIB

Waktu Akhir

Jumat, 9 Oktober 2026, 23.59 WIB

Changelog

Belum ada.

Milestone 1: Sistem Terdistribusi Dasar



Seluruh problem pada milestone ini dikerjakan di atas **satu sistem yang sama**, yakni platform koordinasi kebencanaan yang menjembatani BMKG, PVMBG, dan BNPB (lihat Deskripsi Tugas pada dokumen utama).

Setiap *problem* dinilai dari dua sisi:

- **Laporan** berisi desain, alasan di balik keputusan yang diambil, dan dokumentasi.
- **Implementasi** proof-of-concept yang menunjukkan problem tersebut benar-benar terselesaikan, bukan sekadar dijelaskan.

Cakupan masalah dibatasi pada **dua instansi sumber (BMKG dan PVMBG)** dan **dua jenis bencana (seismik dan vulkanik)**, dengan **BNPB** sebagai ***node* koordinasi**.

Spesifikasi Data & Kontrak *Service*

Bagian ini mendefinisikan komponen, skema data, dan batasan wajib dari setiap lembaga yang disimulasikan pada Milestone 1.

BMKG dan PVMBG adalah simulasi (*mock service*) yang dibangun sendiri oleh kelompok sesuai kontrak di bawah ini. Tujuannya agar seluruh kelompok mengerjakan target yang setara dan dapat dibandingkan secara adil saat gateway BNPB masing-masing dinilai. Kelompok **tidak diperkenankan mengubah** skema, *endpoint*, maupun karakteristik *service* yang telah ditetapkan di bagian ini. Penambahan di luar hal tersebut diperbolehkan selama tidak mengubah hal-hal yang sudah ditetapkan.

Ketentuan Implementasi

Berlaku untuk setiap problem, sejak problem pertama dikerjakan.

1. Komunikasi antara BMKG, PVMBG, BNPB, dan client wajib melalui panggilan jaringan (REST, gRPC, atau *message broker*). Dilarang memanggil fungsi secara langsung maupun berbagi *library business-logic* lintas *service*.

2. Setiap penyimpanan/*storage* dimiliki oleh tepat satu *Service*. *Service* lain yang membutuhkan data tersebut wajib memintanya melalui *service* API pemilik, bukan dengan terhubung langsung ke penyimpanan yang sama. Ketentuan ini berlaku juga untuk *Canonical Store*: store tersebut dimiliki oleh *Aggregator*, dan *Client-Facing* API membaca melalui API *Aggregator*.

3. Setiap *service* wajib dikemas sebagai container Docker tersendiri (masing-masing dengan Dockerfile) dan seluruh sistem dapat dinyalakan melalui satu perintah orkestrasi, contohnya **docker compose up**.

4. Setiap *service* wajib dapat dijalankan dan dimatikan secara independen, dan diuji dengan kegagalan atau keterlambatan yang benar-benar dipicu saat demo, bukan diasumsikan, dan bukan hanya dinarasikan di laporan.

5. Kredensial yang sah di satu instansi tidak berlaku di instansi lain. Solusi berupa satu akun atau kredensial tunggal yang dapat dipakai untuk mengakses BMKG maupun PVMBG sekaligus dianggap **tidak menyelesaikan persoalan**, karena meniadakan premis otonomi antar instansi.

6. Kredensial, kunci penandatanganan token, dan konfigurasi antar-environment wajib dibaca dari environment variable atau berkas konfigurasi yang tidak ikut ter-commit. Repositori wajib menyertakan **.env.example** berisi daftar variabel beserta nilai contoh. Kredensial yang ter-*hardcode* di kode atau ter-commit di repositori akan mengurangi nilai.

7. Setiap *service* wajib menyediakan endpoint health check, menghasilkan log terstruktur yang memuat **correlation ID** yang diteruskan lintas *service* dalam satu alur permintaan, serta mencatat latensi tiap panggilan keluar.

Model Ingest

Gateway BNPB mengambil data dari kedua instansi dengan pola **polling berkala**: kedua *mock* bersifat pasif dan hanya merespons permintaan, tidak melakukan push ke BNPB. Interval polling ditentukan kelompok dan wajib dapat dikonfigurasi, nilai yang disarankan 2–5 detik.

Ketentuan	**Nilai**
Pola *ingest*	Polling dari BNPB ke *mock* (*mock* tidak melakukan *push*)
Interval *polling*	*Configurable*, disarankan 2–5 detik
Laju kemunculan data baru pada mock	Minimal 1 *event* baru per 10 detik per instansi, dapat dipercepat untuk keperluan *demo*
Data awal	Setiap mock wajib menyediakan seed minimal **20 *record*** historis saat dinyalakan
Port	BMKG **8081**, PVMBG **8082**, BNPB Client-Facing API **8080** (dapat diubah lewat konfigurasi, tetapi wajib didokumentasikan di README)

Endpoint minimum yang wajib disediakan *mock*:

Instansi	**Endpoint**	**Keterangan**
BMKG	GET /seismic-events?since=<timestamp>	Mengembalikan **SeismicEvent** sejak waktu tertentu

BMKG	GET /tsunami-warnings?since=<timestamp>	Endpoint terpisah; korelasi ke **SeismicEvent** lewat **related_event_id** dilakukan oleh Aggregator BNPB
BMKG	GET /health	Health check
PVMBG	GET /volcanic-reports?since=<timestamp>	Mengembalikan **VolcanicReport** sejak waktu tertentu
PVMBG	POST /admin/schema-version	Mengaktifkan/menonaktifkan kemunculan **confidence_level** saat sistem berjalan
PVMBG	POST /admin/outage	Menyalakan/mematikan kondisi outage secara instan
PVMBG	GET /health	Health check

Kedua mock **wajib memvalidasi kredensial** yang masuk dan mengembalikan 401/403 bila kredensial tidak sesuai atau berasal dari instansi lain. Mock yang menerima kredensial apa pun membuat Problem 3 tidak dapat dinilai.

Kontrak BMKG (Simulasi)

Domain: pemantauan gempa dan potensi tsunami.

Pada BMKG, terdapat **Entitas SeismicEvent dan Entitas TsunamiWarning**. Entitas **TsunamiWarning** digunakan hanya untuk event dengan **potential_tsunami = true**, diambil melalui endpoint terpisah.

BMKG wajib memiliki *response time rendah* dan konsisten (delay tetap 50–150 ms). Untuk akses ke *service* ini, sertakan API key sendiri melalui header **X-BMKG-Key**.

SeismicEvent

Field	**Tipe**	**Keterangan**
event_id	string	Identifier unik
magnitude	float	Skala Mw
depth_km	float	Kedalaman episentrum
epicenter_lat, epicenter_lon	float	Koordinat episentrum
region_name	string	Nama wilayah terdampak
occurred_at	datetime (ISO 8601)	Waktu kejadian
potential_tsunami	boolean	Penanda potensi tsunami

TsunamiWarning

Field	**Tipe**	**Keterangan**
warning_id	string	Identifier unik
related_event_id	string	Referensi ke **SeismicEvent**

threat_level	enum (**Waspada**/**Siaga**/**Awas**)	Tingkat ancaman
affected_zones	list<string>	Daftar wilayah terdampak
estimated_arrival	datetime	Estimasi waktu tiba gelombang

Kontrak PVMBG (Simulasi)

Domain: pemantauan gunung api.

Karakteristik *service*:

- Response time lebih tinggi dan fluktuatif dibanding BMKG (delay acak 500 ms–3 detik), mensimulasikan infrastruktur lawas. Besaran delay wajib dapat dikonfigurasi.
- Wajib memiliki mekanisme *outage* yang dapat dinyalakan dan dimatikan **secara instan** melalui **POST /admin/outage**. Dalam kondisi outage, PVMBG tidak merespons sama sekali atau mengembalikan galat, menyerupai *service* yang mati. Durasi outage dikendalikan penuh oleh operator demo, sehingga skenario "mati 10–20 menit" dapat ditunjukkan.

Akses ke *service* PVMBG wajib menggunakan kredensial yang terpisah dari BMKG dengan format berbeda, yaitu header **Authorization: Bearer <pvmbg-token>** dengan struktur token tersendiri.

VolcanicReport

Field	Tipe	Keterangan
report_id	string	Identifier unik

volcano_id	string	Identifier gunung api
alert_level	enum (**Normal/Waspada/Siaga/Awas**)	Tingkat aktivitas
eruption_count_24h	integer	Jumlah letusan dalam 24 jam terakhir
ash_column_height_m	float	Tinggi kolom abu (meter)
reported_at	datetime (ISO 8601)	Waktu laporan
confidence_level *(field baru)*	float (0–1)	Muncul di tengah masa berjalan; lihat ketentuan di bawah

Perhatikan bahwa terdapat skenario khusus pada PVMBG. Field **confidence_level** tidak tersedia saat sistem pertama dinyalakan, dan mulai muncul pada respons PVMBG setelah endpoint **POST /admin/schema-version** dipanggil tanpa *mock* maupun gateway di-restart. Urutan berikut wajib didemokan langsung: sistem dinyalakan → gateway bekerja normal dengan skema lama → perubahan skema dipicu → gateway tetap bekerja tanpa restart.

Format Kanonik

Format kanonik adalah representasi tunggal yang dipakai BNPB untuk menyatukan data dari kedua instansi. Seluruh problem yang menyebut "data kanonik" atau "event kanonik" merujuk ke entitas berikut.

Entitas HazardEvent:

Field	Tipe	Klasifikasi	Keterangan
hazard_id	string	Ringkasan	Identifier kanonik yang dibangkitkan BNPB
source	enum (**BMKG/PVMBG**)	Ringkasan	Instansi asal data
source_ref_id	string	Mentah	Identifier asli di instansi sumber (**event_id/report_id**)
hazard_type	enum (**SEISMIC/VOLCANIC**)	Ringkasan	Jenis bahaya
severity	enum (**NORMAL/WASPADA/SIAGA/AWAS**)	Ringkasan	Tingkat ancaman terpadu
area_name	string	Ringkasan	Nama wilayah atau gunung api terdampak
latitude, longitude	float	**Mentah**	Koordinat presisi
occurred_at	datetime (ISO 8601)	Ringkasan	Waktu kejadian di sumber
ingested_at	datetime (ISO 8601)	Ringkasan	Waktu data diterima BNPB

attributes	map<string, any>	**Mentah**	Atribut spesifik sumber yang tidak punya padanan kanonik

Aturan pemetaan wajib dari SeismicEvent -> HazardEvent:

Kanonik	**Asal**
source_ref_id	**event_id**
hazard_type	**SEISMIC**
area_name	**region_name**
latitude, longitude	**epicenter_lat, epicenter_lon**
occurred_at	**occurred_at**
severity	**AWAS** bila terdapat **TsunamiWarning** dengan **threat_level = Awas**; selain itu mengikuti **threat_level** dari **TsunamiWarning** terkait; bila tidak ada warning, **NORMAL** untuk **magnitude < 5.0**, **WASPADA** untuk **5.0 ≤ magnitude < 6.5**, dan **SIAGA** untuk **magnitude ≥ 6.5**
attributes	**magnitude**, **depth_km**, **potential_tsunami**, serta field **TsunamiWarning** terkait bila ada

VolcanicReport -> HazardEvent:

Kanonik	Asal
source_ref_id	**report_id**
hazard_type	**VOLCANIC**
area_name	Nama gunung api hasil pemetaan dari **volcano_id**
latitude, longitude	Koordinat gunung api dari tabel referensi statis milik BNPB
occurred_at	**reported_at**
severity	**alert_level** dipetakan langsung ke enum kanonik
attributes	**eruption_count_24h**, **ash_column_height_m**, serta **confidence_level** bila tersedia

Bila tidak dapat dipetakan/tidak ada padanan kanonik, maka *field* tersebut, termasuk field baru yang muncul kemudian seperti **confidence_level** masuk ke **attributes**.

Kontrak BNPB (Komponen yang Dibangun Kelompok)

BNPB akan menjadi **objek utama** yang dinilai.

Komponen	Fungsi	Terkait Problem
Gateway / Aggregator	Melakukan polling ke BMKG dan PVMBG, memetakan data ke **HazardEvent**, dan mempublikasikan event	P1, P2, P5

Canonical Store	Menyimpan **HazardEvent**; dimiliki dan hanya diakses langsung oleh Aggregator	P1, P4
Client-Facing API	Endpoint yang diakses client downstream	P2, P3
Auth / Token Service	Menerbitkan token ke client downstream dengan scope berbeda	P3
Message Broker	Menyalurkan **HazardEvent** dari producer ke banyak consumer	P5
Event Consumer (≥2)	Berlangganan **HazardEvent**, contohnya dashboard updater dan notifier	P5

Client Downstream & Hak Akses

Client	**Hak Akses**	**Catatan**
Media / Pers	Hanya field berklasifikasi **Ringkasan**	Kebocoran field mentah termasuk koordinat presisi dan isi **attributes** dianggap gagal
Tim Lapangan (BASARNAS / Pemda)	Seluruh field, termasuk **Mentah**	Access token ber-TTL pendek; wajib tersedia alur refresh tanpa login ulang manual

BNPB Pusat / Internal Ops	Seluruh field, termasuk **Mentah**	Baseline hak akses tertinggi

Requirements:

1. Ketiga client wajib diimplementasikan sebagai identitas dan kredensial yang benar-benar berbeda, bukan satu akun dengan flag role di sisi client. Pembeda hak akses ditegakkan di sisi BNPB, bukan diasumsikan dipatuhi client.
2. **Access token** wajib ber-TTL pendek dan dapat dikonfigurasi, dengan nilai default **60 detik**, agar skenario kedaluwarsa dapat didemokan tanpa menunggu lama atau mengubah jam sistem.
3. Pada M1, seluruh client bersifat **read-only**. Jalur pelaporan dari lapangan ke BNPB tidak termasuk cakupan M1.

Problem Set

Problem 1

Arsitektur & Interoperabilitas Data



Prototipe pertama gateway BNPB yang dibangun tim Astolfo bekerja dengan asumsi sederhana: field-field **SeismicEvent** dari BMKG dan **VolcanicReport** dari PVMBG dianggap tetap selamanya, sehingga parsing-nya ditulis dengan mengasumsikan struktur itu tidak akan pernah berubah. Semuanya baik-baik saja, sampai suatu hari PVMBG menambahkan **confidence_level** ke laporan mereka tanpa memberi tahu siapa pun terlebih dahulu bagi PVMBG itu perubahan internal mereka sendiri, bukan urusan BNPB. Gateway Astolfo, yang tidak siap menerima field baru, mulai berperilaku tidak terduga: entah field itu hilang begitu saja, entah keseluruhan pesan gagal di-parse.

Gateway di-*hardcode* terhadap bentuk skema yang diasumsikan tetap, sehingga perubahan skema aditif dari salah satu instansi memaksa perubahan kode gateway dan *redeploy*. Padahal, instansi sumber tidak berkoordinasi soal kapan mereka boleh mengubah skema.

Kriteria penyelesaian:

1. Jalankan sistem dengan PVMBG pada skema awal, lalu tunjukkan Aggregator menghasilkan **HazardEvent** yang sesuai dengan aturan pemetaan untuk kedua sumber.

2. Picu perubahan skema PVMBG melalui **POST /admin/schema-version** saat sistem sedang berjalan, tanpa restart atau redeploy *servic*e mana pun.

3. Tunjukkan Aggregator tetap menghasilkan **HazardEvent** yang valid setelah perubahan tersebut, tanpa crash dan tanpa intervensi manual pada kode.

4. Deklarasikan kebijakan penanganan field tak dikenal di laporan apakah diteruskan ke **attributes** atau diabaikan lalu tunjukkan perilaku sistem konsisten dengan kebijakan yang dideklarasikan. **Yang dinilai adalah konsistensi terhadap kebijakan yang dinyatakan, bukan pilihan kebijakannya.**

5. Di laporan, jelaskan mekanisme yang dipilih (contoh: schema registry, dynamic field mapping, schema evolution ala Protobuf/Avro, atau JSON dengan kebijakan additive-only), serta nyatakan eksplisit jenis perubahan skema yang didukung dan yang secara sadar tidak didukung.

Problem 2

Concurrency, *Availability*, & *Graceful Degradation*



Begitu gateway dasar Astolfo berjalan, masalah baru muncul pada demo internal. Karena gateway memanggil BMKG lalu PVMBG secara berurutan dalam satu alur yang sama, dan PVMBG dengan infrastruktur lawasnya membutuhkan waktu jauh lebih lama untuk merespons, permintaan yang sebetulnya hanya memerlukan data BMKG pun ikut menunggu tanpa alasan. Ketika status berubah dari normal ke siaga dan banyak pihak mengakses gateway bersamaan, sistem yang tadinya hanya terasa lambat berubah menjadi tidak responsif. Lalu PVMBG benar-benar mati selama beberapa waktu dan seluruh dashboard BNPB ikut kosong, padahal data gempa dari BMKG masih mengalir normal dan data vulkanik terakhir sebenarnya masih tersimpan.

Terdapat tiga kegagalan berbeda:

1. **Head-of-line blocking**
   Pemanggilan BMKG dan PVMBG yang sekuensial membuat permintaan cepat tertahan oleh permintaan lambat yang tidak berhubungan.
2. **Resource exhaustion**

Tidak ada batas jumlah permintaan bersamaan yang dapat ditangani, sehingga lonjakan akses menghabiskan resource hingga sistem berhenti melayani semua pihak.

3. **Ketiadaan degradasi terkendali**
   Matinya satu sumber membuat seluruh respons gagal, padahal sebagian data masih tersedia dan masih berguna bagi pengguna.

Kriteria penyelesaian:

1. Dengan delay PVMBG diatur pada 3 detik, kirim permintaan yang hanya membutuhkan data BMKG bersamaan dengan permintaan yang membutuhkan data PVMBG. Tunjukkan **p95 latensi permintaan BMKG-only tetap di bawah 300 ms**.

2. Jalankan load testing dengan **≥50 koneksi paralel selama minimal 60 detik** dalam pola sustained. Laporkan throughput, p50/p95/p99, dan error rate. Sistem wajib tidak crash, dengan **error rate di luar penolakan terkontrol di bawah 1%**. Penolakan terkontrol seperti HTTP 429 tidak dihitung sebagai kegagalan, selama jumlahnya dilaporkan.

3. Nyalakan outage PVMBG melalui **POST /admin/outage**. Selama outage, permintaan data seismik tetap dilayani normal, permintaan data vulkanik dijawab dengan data terakhir dari Canonical Store disertai penanda kebaruan data (contoh: **stale_since**) atau dengan status ketidaktersediaan yang eksplisit **bukan** error 500 dan bukan permintaan yang menggantung sampai timeout client. Setelah outage dimatikan, tunjukkan sistem kembali normal tanpa restart.

4. Di laporan, jelaskan model konkurensi yang dipilih dan bagaimana model tersebut mencegah ketiga kegagalan di atas. Sertakan konfigurasi *load testing* beserta *tooling* yang dipakai.

Problem 3

Service-to-Service Authentication & Trust



Versi awal sistem Astolfo memakai satu kunci rahasia yang sama untuk mengakses BMKG maupun PVMBG, sekaligus dipakai ulang sebagai kunci akses bagi client di luar BNPB. Rasanya begitu praktis, sampai tim keamanan mengajukan pertanyaan sederhana: kalau kunci milik media bocor, apakah mereka juga bisa melihat data mentah? Jawabannya iya.

Tidak ada pemisahan *domain of trust*. Kredensial ke BMKG, kredensial ke PVMBG, dan token untuk *client downstream* seharusnya independen. Kompromi pada satu domain tidak boleh berdampak ke domain lain.

Kriteria penyelesaian:

1. Tunjukkan kredensial yang valid untuk BMKG ditolak dengan **401/403** saat dipakai mengakses PVMBG, dan sebaliknya.
2. Tunjukkan client ber-scope Media hanya menerima field berklasifikasi Ringkasan. Permintaan eksplisit terhadap field mentah ditolak server dengan **401/403**, bukan sekadar disembunyikan di sisi tampilan.
3. Tunggu access token Tim Lapangan kedaluwarsa secara alami sesuai TTL (default 60 detik) di tengah sesi aktif. Tunjukkan alur refresh menghasilkan token baru tanpa login ulang manual, dan tunjukkan token lama ditolak setelah refresh.

4. Tunjukkan tidak ada kredensial yang ter-*hardcode* di kode maupun ter-commit, dan **.env.example** tersedia sesuai Ketentuan Implementasi.
5. Di laporan, jelaskan mekanisme yang dipakai beserta alasan mengapa mekanisme tersebut tidak membuka celah baru.

Problem 4

Arsitektur Microservice & *Flexible Storage*



Gateway Astolfo kini benar secara fungsi, tetapi tim mulai kesulitan mengoperasikannya. Seluruh logic polling, pemetaan skema, autentikasi, dan penyajian ke client masih hidup dalam satu unit deploy yang sama, sehingga satu perbaikan kecil di bagian autentikasi memaksa seluruh sistem ikut di-deploy ulang, termasuk bagian pemetaan skema yang sama sekali tidak berubah. Masalah lain menyusul dari sisi penyimpanan: **HazardEvent** disimpan pada tabel relasional dengan kolom yang ditetapkan di muka, sehingga setiap kali PVMBG menambah field baru seperti **confidence_level**, tim harus mengubah definisi tabel dan menjalankan migrasi yang sekali lagi memaksa koordinasi dan deploy bersama, persis masalah yang sejak awal ingin dihindari.

Dua hal yang saling berkaitan:

1. Sistem belum berbentuk kumpulan *service* yang independen, terbukti dari kenyataan bahwa mengubah satu bagian memaksa deploy ulang bagian lain yang tidak berhubungan.

2. Mekanisme penyimpanan tidak sejalan dengan sifat data yang skemanya dapat berubah sewaktu-waktu, sehingga masalah coupling dari Problem 1 kembali muncul melalui jalur *storage*.

Kriteria penyelesaian:

1. Tunjukkan setiap komponen berjalan sebagai container terpisah, dan tunjukkan salah satunya dapat dihentikan, di-rebuild, dan dijalankan ulang **tanpa** me-rebuild atau merestart container lain. Tunjukkan pula sistem tetap melayani permintaan yang tidak bergantung pada komponen yang sedang di-restart.

2. Simpan **HazardEvent** pada penyimpanan yang mengakomodasi atribut dinamis tanpa migrasi skema saat field baru muncul. Pilihan teknologi bebas document store, key-value store, maupun basis data relasional dengan kolom dokumen seperti JSONB selama tidak ada langkah migrasi yang diperlukan ketika **confidence_level** mulai muncul. Tunjukkan record dari sebelum dan sesudah perubahan skema tersimpan berdampingan dan keduanya dapat dibaca.

3. Tunjukkan Canonical Store hanya dapat diakses langsung oleh Aggregator, dan komponen lain memperoleh data melalui API Aggregator, sesuai ketentuan poin di atas (2) yang anda implementasikan.

4. Di laporan, jelaskan alasan penentuan batas tiap *service,* mengapa dipecah seperti itu dan bukan dengan pembagian lain. Serta, bandingkan minimal dua alternatif penyimpanan beserta konsekuensi dari pilihan yang diambil.

Problem 5

Fan-out Event Distribution/Pub-sub



Semakin sistem dipakai, semakin banyak pihak yang ingin tahu setiap kali ada data baru masuk. Tim dashboard internal ingin update otomatis begitu ada **HazardEvent** baru, tim notifikasi ingin segera mengirim peringatan ke perangkat tim lapangan, dan belakangan muncul rencana menyambungkan *feed* yang sama ke portal pemerintah daerah. Jika Aggregator harus memanggil setiap pihak satu per satu secara langsung, satu pihak yang lambat dapat menahan pengiriman notifikasi ke tim lapangan yang jauh lebih mendesak. Setiap kali ada pihak baru yang ingin berlangganan pun, kode Aggregator harus diubah lagi.

Producer dan consumer saling terikat langsung. Producer harus mengetahui dan memanggil tiap consumer secara sinkron, sehingga kelambatan atau kegagalan satu consumer berdampak pada producer maupun consumer lain, dan penambahan consumer baru berarti mengubah kode producer.

Kriteria penyelesaian:

1. Setiap **HazardEvent** baru hasil pemetaan dipublikasikan ke message broker (contoh: Kafka, RabbitMQ, NATS), bukan dikirim langsung ke masing-masing consumer.

2. Tunjukkan minimal dua consumer independen berlangganan pada stream yang sama dan menerima event tanpa producer mengetahui keberadaan mereka.

3. Hentikan salah satu consumer selama beberapa waktu. Tunjukkan producer tetap mempublikasikan event tanpa tertahan dan consumer lain tetap menerima event normal. Setelah consumer dinyalakan kembali, jelaskan dan tunjukkan apa yang terjadi pada event yang terbit selama consumer mati apakah tersusul atau hilang sesuai konfigurasi broker yang dipilih.

4. Tunjukkan penambahan satu consumer baru pada *topic* yang sudah ada, tanpa mengubah kode producer sama sekali.

5. Di laporan, jelaskan broker dan pola yang dipilih, semantik pengiriman yang berlaku (at-least-once atau at-most-once), serta konsekuensinya bagi consumer termasuk apakah consumer perlu bersifat idempoten dan bagaimana kelompok menyikapinya.

Deliverables

1. **Laporan** dalam format PDF, sesuai Bagian 5 (Laporan).
2. **Repositori kode** berisi seluruh *mock*, seluruh komponen BNPB, *orchestration files*, **.env.example**, dan README.

Laporan

Laporan dikumpulkan dalam format PDF dengan format **IF4031_M1_<NamaKelompok>.pdf.**

Bagian Umum

1. **Halaman sampul** yang berisis nama kelompok, nama dan NIM setiap anggota, nama sistem, serta identitas mata kuliah.
2. **Tabel kontribusi** yang menyajikan kontribusi setiap anggota terhadap perancangan, implementasi, dan penulisan laporan.
3. **Deskripsi umum sistem** berisi gambaran ringkas sistem dan lingkup yang dicakup M1.
4. **Diagram arsitektur sistem** yang mencakup seluruh *service*, protokol komunikasi antar *service*, kepemilikan penyimpanan, alur event, dan batas container. Wajib disertai penjelasan naratif.
5. **Daftar teknologi terpilih** berisi teknologi tiap komponen beserta alasan pemilihan yang dikaitkan dengan kebutuhan sistem.
6. **Asumsi** berisi seluruh asumsi yang diambil dalam perancangan maupun implementasi.
7. **Petunjuk menjalankan sistem** berupa ringkasan yang merujuk ke README.

Bagian Per Problem

Untuk setiap problem (P1–P5):

1. **Rancangan solusi** dan mekanisme yang dipilih.
2. **Alasan keputusan** berupa alternatif yang dipertimbangkan dan trade-off dari pilihan yang diambil.
3. **Batasan yang disadari** menjelaskan hal yang secara sadar tidak ditangani, beserta alasannya.
4. **Bukti penyelesaian** berupa tangkapan layar, potongan log ber-correlation ID, hasil pengukuran, atau keluaran load testing yang menunjukkan tiap kriteria terpenuhi.
5. **Jawaban atas butir "Di laporan"** pada problem terkait.

Ketentuan Penulisan

- Setiap klaim harus dapat ditelusuri ke bukti pada dokumentasi atau ke kode pada repositori.
- Penggunaan LLM diperbolehkan dengan syarat dideklarasikan eksplisit dan kode yang dihasilkan dipahami sepenuhnya oleh kelompok.
- Kesamaan bagian laporan atau kode dengan pekerjaan kelompok lain maupun repositori di internet dianggap kecurangan, dan alasan penggunaan LLM tidak diterima.

Demonstrasi

Demonstrasi akan dilakukan secara **sinkron** setelah tenggat *milestone* 1. Detail demonstrasi akan diberikan paling lambat **H+1** dari tenggat *milestone* 1.

Pengumpulan

Seluruh deliverables dikumpulkan sebelum **Jumat, 9 Oktober 2026, pukul 23:59 WIB**. Tenggat bersifat ***hard deadline***.

Mekanisme:

1. **Repositori** dibuat di GitHub dengan visibilitas **private**, dan akses dijadikan **public** setelah deadline.
2. Kelompok membuat **tag** bernama **milestone-1** pada commit terakhir yang hendak dinilai. Commit setelah tenggat tidak dinilai.
3. Formulir pengumpulan diisi satu kali per kelompok, memuat: nama kelompok, NIM pengumpul, tautan repositori, dan tautan laporan PDF.
4. **Penamaan berkas laporan: IF4031_M1_<NamaKelompok>.pdf**.

README **wajib** memuat langkah menjalankan sistem dari kondisi awal, daftar variabel konfigurasi, serta cara memicu setiap skenario pengujian. Kode yang tidak dapat dijalankan mengikuti README hanya dinilai seadanya.

Kelompok diperbolehkan mengadakan asistensi dengan berkoordinasi langsung bersama asisten.

Penilaian

Komponen	**Poin**
Problem Set (5 problem × 18 poin)	90
Demonstrasi	10
Total	**100**

Setiap problem bernilai 18 poin, terbagi rata:

Sisi Penilaian	**Poin per Problem**	**Total**
Laporan	9	45
Implementasi	9	45



“Good luck bos!”